

SBA301 – INTEGRATE SINGLE PAGE APPLICATION WITH SPRING BOOT

SLOT 6

REACT HOOK

[useState](#) • [useEffect](#) • [useContext](#) • [useRef](#) • [Custom Hooks](#) • [Rules of Hooks](#)

Giáo trình bài giảng & hướng dẫn tự học dành cho sinh viên

Định hướng: Project-Based Learning • Human Verification • Research-Based Learning • AI-Augmented Learning

Thông tin	Nội dung
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	6
Chủ đề	Chapter 05 – React Hook + liên hệ Lab 01
Công nghệ	ReactJS, JavaScript/JSX, Node 21 theo baseline môn học, Vite/CRA theo tài nguyên lớp
Đối tượng	Sinh viên Bachelor; dùng trên lớp, tự học, thực hành và ôn tập
Kết quả trọng tâm	Hiểu bản chất Hook và sử dụng đúng useState, useEffect, useContext, useRef, Custom Hook

Mental model trung tâm — Hook không chỉ là “cú pháp mới”. Hook là cơ chế để function component sử dụng và tổ chức state, context, ref, effect và logic tái sử dụng một cách có quy tắc.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT 6

Mục tiêu chuẩn bị: bảo đảm sinh viên không bước vào Hook như một khái niệm rời rạc. Hook phải được nối trực tiếp với component, props, state, context và event-driven UI đã học trước đó.

0.1 Kiến thức cần ôn lại

Nội dung	Sinh viên cần nhớ	Câu hỏi tự kiểm tra
JSX & Component	Component nhận input và mô tả UI bằng JSX.	Component khác DOM element như thế nào?
Function Component	Hàm JavaScript trả về JSX/React elements.	Vì sao tên component viết hoa?
React-Bootstrap	UI component library dùng trong các slot trước.	Library UI có quản lý state thay ứng dụng không?
Props	Dữ liệu cha truyền xuống con, read-only đối với bên nhận.	Ai sở hữu props?
State	Bộ nhớ cục bộ ảnh hưởng UI, cập nhật qua setter.	Vì sao biến thường không đủ?
Lifting State Up	Đưa state lên ancestor chung khi nhiều con cần phối hợp.	Ai là owner phù hợp?
Context	Chia sẻ dữ liệu đến descendant khi props drilling thực sự trở nên khó.	Context có phải database không?
map() & Event	Render collection và xử lý click/change.	map() có phải Hook không?

0.2 Kỹ năng tối thiểu

- Mở project React hiện có bằng IntelliJ/IDE và xác định đúng thư mục chứa package.json.
- Chạy được npm run dev và kiểm tra Browser Console.
- Tạo function component, import/export component.
- Truyền props cha → con; đọc object prop.
- Dùng useState cơ bản và xử lý onClick/onChange.
- Render danh sách bằng map() với key ổn định.

0.3 Công cụ & môi trường

Công cụ	Vai trò	Kiểm tra nhanh
Node 21 (baseline môn học)	Runtime/tooling JavaScript theo học liệu lớp.	node -v
npm/npx	Quản lý package và script.	npm -v
IntelliJ IDEA / IDE tương đương	Viết code, Terminal, Run Configuration.	Mở package.json; Node interpreter đúng.
Chrome/Edge	Chạy và debug ứng dụng.	Console không có blocking error.
React Developer Tools	Quan sát component tree, props/state.	Extension hoạt động trong DevTools.
Git/GitHub	Checkpoint bằng chứng và cộng tác.	git status

Lưu ý Vite — Slot 6 nên tiếp tục trên project đã chạy ổn từ các slot trước. Nếu phải scaffold mới, dùng phiên bản Vite/Node đã được giảng viên khóa cho lớp; không tự nâng tooling giữa buổi học chỉ để chạy “latest”.

0.4 Tài liệu đọc trước & bài chuẩn bị

- Slide Ch05 1–19.
- React.dev – Built-in React Hooks / useState / useEffect / useContext / useRef.
- React.dev – State: A Component's Memory.
- React.dev – You Might Not Need an Effect.
- Tài liệu Slot 5 – Props, State and Context.
- Lab 01 – Working with ReactJS application.

Bài chuẩn bị 3-2-1: ghi 3 khái niệm về state/props, 2 thí nghiệm nhỏ (Counter và controlled input), 1 câu hỏi còn chưa chắc về state hoặc side effect.

0.5 Pre-slot Human Check (không dùng AI)

Câu hỏi	Tự trả lời
Props và state khác nhau như thế nào?	_____
Khi nào cần state?	_____
Một biến thường thay đổi có làm React render lại không?	_____
Context giải quyết vấn đề gì?	_____
State owner nên được đặt ở đâu?	_____
Side effect theo suy đoán của bạn là gì?	_____

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Bảng tóm tắt kiến thức Slot 5

Nhóm kiến thức	Nội dung chính	Ý nghĩa	Ví dụ	Vấn đề Hook giúp giải quyết
Props	Cha truyền dữ liệu xuống con; bên nhận không sửa props.	Làm dependency giữa component rõ ràng.	<ProductCard product={p} />	Không giải quyết bộ nhớ cục bộ hoặc side effect.
State	Dữ liệu component “nhớ” và ảnh hưởng UI.	Cho UI phản ứng với tương tác.	isOpen, count, searchText	Hook cung cấp API state cho function component.
Lifting state up	Đưa state lên ancestor chung.	Đồng bộ nhiều component con.	Filter + ProductList	Custom hook giúp tái sử dụng logic; vẫn phải xác định owner.
Context	Chia sẻ giá trị tới descendant.	Giảm prop drilling khi thực sự cần.	theme/currentUser	useContext giúp đọc/subscribe context trong function component.
Controlled form	value đến từ state; onChange cập nhật state.	Single source of truth cho input.	name + setName	useState là nền tảng.
One-way data flow	Data đi từ owner đến consumer.	Dễ dự đoán và debug.	Parent → props → child	Hooks không phá vỡ nguyên tắc ownership.
Re-render	State/props/context thay đổi có thể làm UI tính lại.	UI phản ánh dữ liệu mới.	setCount(c=>c+1)	Cần hiểu dependency/effect để tránh loop.
Chia sẻ dữ liệu	Props, lifting state, Context.	Chọn scope phù hợp.	currentUser	useContext và custom hooks giúp tổ chức logic.

2. Bảng định vị Slot 6 với các slot kế tiếp

Slot/Phần	Chủ đề	Mối liên hệ với Slot 6	Kiến thức Hook hỗ trợ
Slot 5	Props, State, Context	Nền tảng data/state ownership.	useState/useContext làm API cụ thể cho function component.
Slot 6	React Hook + Lab 01	Tổ chức state, side effect, ref, shared logic.	useState, useEffect, useContext, useRef, custom hook.
Slot 7	Handling Navigation with Routes	Router hooks và navigation state dựa trên mental model Hook.	Tư duy hook + state giúp hiểu useParams/useNavigate sau đó.
Slot 8	Client–Server Communication	HTTP làm phát sinh async data, loading, error.	useEffect awareness chuẩn bị cho synchronization; sau này fetch/service.
Slot 9	Fetching & Caching Data	Quản lý server data phức tạp hơn local state.	Hiểu effect, state, custom hooks để đánh giá caching library.
Slot 10	Router/SPA mở rộng	App có nhiều view và state dùng chung.	Hook hỗ trợ auth/session/form state và integration.
Lab 01	Working with ReactJS application	Nơi kiểm chứng các kỹ năng bằng artifact.	Hook tạo tương tác, effect, theme, focus, logic tái sử dụng.

3. Bối cảnh bài học

Ở Slot 5, sinh viên đã biết “data là gì” và “ai sở hữu data”. Slot 6 trả lời câu hỏi tiếp theo: function component làm thế nào để nhớ dữ liệu, đồng bộ với thế giới bên ngoài, đọc dữ liệu dùng chung và giữ một tham chiếu không cần render lại? Hook là lớp công cụ giải quyết các nhu cầu đó, nhưng chỉ hiệu quả khi sinh viên hiểu đúng ownership và side effect.

Current React practice — React.dev hiện mô tả Effects là một “escape hatch” để đồng bộ component với external system. Nếu không có external system, trước hết hãy kiểm tra logic có thể tính trực tiếp trong render hoặc xử lý trong event handler thay vì dùng useEffect.

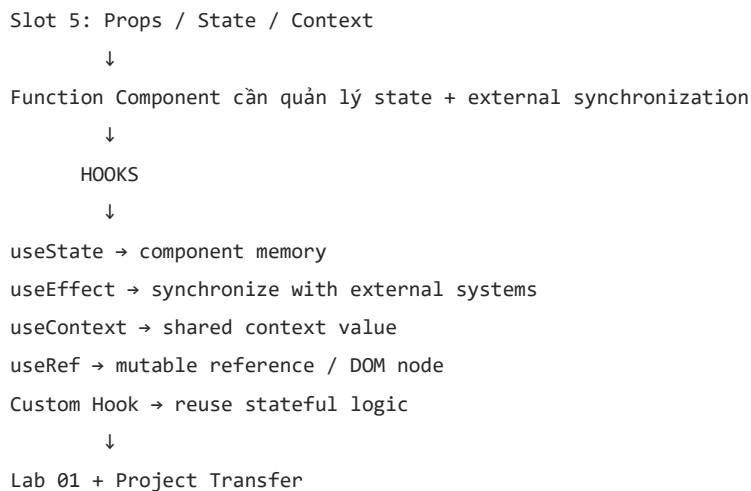
4. Mục tiêu học tập

- Giải thích React Hook là gì và vì sao ra đời.
- So sánh Function Component trước Hook, Class Component và Function Component với Hook.
- Dùng useState cho number/string/object/array và controlled input.
- Dùng useEffect cho side effect thật sự; phân tích dependency array và cleanup.
- Dùng useContext đúng scope; tránh coi Context là “global state mặc định”.
- Dùng useRef cho DOM reference hoặc giá trị không cần re-render.
- Tạo custom hook đơn giản để tái sử dụng logic stateful.
- Tuân thủ Rules of Hooks và nhận diện vi phạm.
- Vận dụng vào Lab 01 và giải thích được dữ liệu/effect/ref của chính mình.

5. Khái niệm trung tâm

Khái niệm	Cốt lõi
Hook	Hàm đặc biệt cho phép React function component sử dụng các khả năng React như state, context, refs, effects.
useState	Bộ nhớ cục bộ của component có setter gây cập nhật UI.
useEffect	Đồng bộ component với external system sau commit; có setup/dependencies/cleanup.
Dependency array	Danh sách reactive values Effect phụ thuộc; quyết định khi setup/cleanup cần chạy lại.
Cleanup	Hoàn tác/giải phóng resource trước lần Effect kế tiếp hoặc khi unmount.
useContext	Đọc và subscribe vào Context gần nhất.
useRef	Giữ mutable reference không dùng để render; thay đổi ref không tự gây re-render.
Custom Hook	Hàm bắt đầu bằng use, kết hợp Hooks để tái sử dụng stateful logic.
Rules of Hooks	Gọi Hook top-level và chỉ từ React function/custom Hook.
Side effect	Tương tác/synchronization với external system: network, timer, browser API, subscription, DOM ngoài luồng render.

6. Flow tổng quát của Slot 6



7. Mini demo app trong slot

Mini demo	Hook chính	Mục tiêu
Counter	useState	State update + re-render
Controlled Name Input	useState	value/onChange + single source of truth
Document Title	useEffect	External browser API synchronization
Timer	useEffect + useRef	setup/cleanup + interval id
Theme	useContext	Shared data from provider
Focus Search Input	useRef	DOM reference
useCounter	Custom Hook	Reuse stateful logic

8. Tư duy kiến trúc cần giữ

- Giá trị này có cần “nhớ” giữa các render và ảnh hưởng UI không? → cân nhắc state.
- Đây có phải external system/synchronization không? → cân nhắc Effect.
- Giá trị cần chia sẻ qua nhiều tầng không? → cân nhắc Context.
- Giá trị không dùng để render nhưng cần giữ reference? → cân nhắc ref.
- Logic stateful này có lặp ở nhiều component không? → cân nhắc custom hook.

9. Timeline gợi ý 135 phút

STT	Hoạt động
1.	Recall Slot 5 + Pre-check
2.	Why Hooks + Function vs Class
3.	useState + controlled input
4.	useEffect + dependency + cleanup
5.	useContext
6.	useRef
7.	Custom Hooks + Rules of Hooks
8.	Lab 01 integration / debugging
9.	AI/Research + project transfer
10.	Human Verification + exit check

10. Tài liệu tham khảo

- React.dev – Built-in React Hooks: <https://react.dev/reference/react/hooks>
- React.dev – Rules of Hooks: <https://react.dev/reference/rules/rules-of-hooks>
- React.dev – State: A Component’s Memory: <https://react.dev/learn/state-a-components-memory>
- React.dev – useEffect: <https://react.dev/reference/react/useEffect>
- React.dev – You Might Not Need an Effect: <https://react.dev/learn/you-might-not-need-an-effect>
- React.dev – Reusing Logic with Custom Hooks: <https://react.dev/learn/reusing-logic-with-custom-hooks>
- W3Schools – React Hooks (supplementary beginner reference).
- Slide Ch05 1–19 và Lab 01 của môn SBA301.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. React Hook là gì?

Hook là một nhóm hàm theo quy ước React, thường bắt đầu bằng “use”, cho phép function component sử dụng các khả năng React như state, context, ref và effect. Hook không phải component và không trả về UI chỉ vì nó là Hook; component mới là đơn vị render UI.

Điểm cần nhớ — Hook không thay thế component. Hook tổ chức logic liên quan React bên trong function component hoặc custom hook.

2. Vì sao cần Hook?

- Lịch sử: function component ban đầu chủ yếu mô tả UI từ props; state/lifecycle thường gắn với class component.
- Class component có this, this.state, this.setState và lifecycle methods; cùng một concern có thể bị tách giữa nhiều lifecycle method.
- Hook cho phép function component quản lý state/effect/context/ref mà không cần class.
- Custom Hook giúp tái sử dụng stateful logic giữa component mà không copy/paste implementation.
- Hook làm logic gắn với feature/use-case hơn, nhưng vẫn cần thiết kế state ownership và dependency đúng.

3. Function Component vs Class Component

Tiêu chí	Function trước Hook (bối cảnh lịch sử)	Class Component	Function + Hook
Cú pháp	Hàm nhận props, trả UI.	class extends React.Component + render().	Hàm + Hook.
State	Không có API Hook thời kỳ trước 16.8.	this.state / this.setState.	useState/useReducer.
Side effect/lifecycle	Không có Hook effect.	componentDidMount/Update/WillUnmount.	useEffect và các Hook khác.
this binding	Không.	Có thể phải xử lý this.	Không dùng this cho component logic.
Reuse logic	Pattern như HOC/render props.	HOC/render props/mixins lịch sử.	Custom Hooks.
Độ phức tạp	Đơn giản nhưng hạn chế.	Cao hơn với lifecycle/this.	Thường gọn hơn khi tổ chức tốt.
React hiện đại	Bối cảnh lịch sử.	Vẫn tồn tại trong legacy code.	Ưu tiên cho code mới.

Ví dụ Counter – Class Component

```
class Counter extends React.Component {
  state = { count: 0 };

  increase = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increase}>Increase</button>
      </div>
    );
  }
}
```

Ví dụ Counter – Function Component + Hook

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>Increase</button>
    </div>
  );
}
```

Điểm so sánh: function component + Hook không dùng this; state logic nằm gần event/UI. Tuy nhiên “ít dòng code” không tự động đồng nghĩa kiến trúc tốt: state owner, effect dependencies và naming vẫn phải rõ ràng.

4. useState – Component Memory

useState dùng khi component cần nhớ một giá trị giữa các lần render và khi giá trị đó thay đổi thì UI cần được tính lại.

```
const [value, setValue] = useState(initialValue);
```

Thành phần	Ý nghĩa
value	Snapshot state của lần render hiện tại.
setValue	Yêu cầu React cập nhật state và schedule render mới.

Thành phần	Ý nghĩa
initialValue	Giá trị khởi tạo ở lần đầu.

Counter với functional update

```
const [count, setCount] = useState(0);

function increase() {
  setCount(c => c + 1);
}

function decrease() {
  setCount(c => c - 1);
}

function reset() {
  setCount(0);
}
```

Best practice — Khi state mới phụ thuộc trực tiếp vào state trước, ưu tiên functional updater như `setCount(c => c + 1)`.

Controlled input

```
const [name, setName] = useState('');

<input
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
<p>Hello {name || 'student'}</p>
```

Object state – immutable update

```
const [profile, setProfile] = useState({
  username: '',
  email: ''
});

function updateEmail(e) {
  setProfile(prev => ({
    ...prev,
    email: e.target.value
  }));
}
```

Array state – add/remove

```
const [products, setProducts] = useState(initialProducts);

function addProduct(newProduct) {
  setProducts(prev => [...prev, newProduct]);
}
```

```
function removeProduct(id) {
  setProducts(prev => prev.filter(p => p.id !== id));
}
```

Lỗi	Vì sao sai	Cách sửa
count = count + 1	Biến snapshot bị gán; React không được yêu cầu cập nhật state.	setCount(c => c + 1)
products.push(x)	Mutate array cũ, khó reasoning/change detection.	setProducts(prev => [...prev, x])
profile.email = x	Mutate object state.	Spread object mới.
Lưu filteredProducts vào state khi có thể tính từ products + query	Tạo duplicated/derived state dễ lệch.	Tính trong render; chỉ state hóa source value.

5. useEffect – Synchronization với external system

useEffect không nên được hiểu đơn giản là “code chạy sau render”. Mental model tốt hơn: Effect mô tả cách component đồng bộ với một hệ thống bên ngoài React, chẳng hạn document API, timer, network, subscription hoặc thư viện không phải React.

```
useEffect(() => {
  // setup / synchronize
  return () => {
    // cleanup / undo previous setup
  };
}, [dependencies]);
```

Dependency array – ba dạng cơ bản

Cú pháp	Mental model	Cảnh báo
useEffect(setup)	Effect được xét sau mỗi commit; setup/cleanup có thể chạy nhiều lần.	Rất dễ dư thừa nếu setup cập nhật state.
useEffect(setup, [])	Effect không phụ thuộc reactive value thay đổi; thường tương ứng “mount/unmount” về mặt học nhập môn.	Trong development StrictMode có thể thấy thêm chu kỳ setup→cleanup→setup.
useEffect(setup, [a,b])	Đồng bộ lại khi dependency liên quan thay đổi.	Không cố tình bỏ dependency để “ép” effect chạy ít.

Ví dụ 1 – document.title

```
const [count, setCount] = useState(0);

useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]);
```

Ở đây external system là browser document title. Khi count thay đổi, Effect đồng bộ title với state.

Ví dụ 2 – interval + cleanup

```
useEffect(() => {
  const id = setInterval(() => {
    setSeconds(s => s + 1);
  }, 1000);

  return () => clearInterval(id);
}, []);
```

Cleanup ngăn interval tiếp tục tồn tại sau khi component không còn cần nó. Trong development StrictMode, chu kỳ setup/cleanup bổ sung giúp lộ lỗi cleanup không đối xứng.

Ví dụ 3 – resize listener + cleanup

```
useEffect(() => {
  function handleResize() {
    setWidth(window.innerWidth);
  }

  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);
```

Ví dụ 4 – Fetch API: preview chuẩn bị Slot 8/9

```
useEffect(() => {
  const controller = new AbortController();

  async function loadUsers() {
    try {
      setLoading(true);
      const res = await fetch('https://jsonplaceholder.typicode.com/users', {
        signal: controller.signal
      });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setUsers(await res.json());
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  loadUsers();
  return () => controller.abort();
}, []);
```

Source boundary — Fetch trong Slot 6 chỉ là demo Effect/async. Client-server communication và data fetching/caching sẽ được học sâu ở các slot sau; không biến Slot 6 thành bài HTTP/REST.

Khi KHÔNG nên dùng useEffect

- Tính filteredProducts từ products + query → tính trực tiếp trong render.
- Xử lý hành động Buy/Add/Delete → làm trong event handler biết rõ sự kiện nào xảy ra.
- Đồng bộ state A từ state B chỉ vì có thể → kiểm tra có thể xóa state A và dùng derived value không.

6. useContext – Đọc dữ liệu dùng chung

useContext đọc và subscribe vào Context gần nhất phía trên component. Context phù hợp với dữ liệu nhiều descendant cần dùng như theme, authenticated user hoặc locale; không phải database, cache server hay thay thế mặc định cho mọi props.

ThemeContext – cấu trúc cơ bản

```
// ThemeContext.jsx
import { createContext, useContext, useState } from 'react';
```

```

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () =>
    setTheme(t => (t === 'light' ? 'dark' : 'light'));

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const value = useContext(ThemeContext);
  if (!value) throw new Error('useTheme must be used inside ThemeProvider');
  return value;
}

function ThemeToggle() {
  const { theme, toggleTheme } = useTheme();
  return <button onClick={toggleTheme}>Theme: {theme}</button>;
}

```

Nên dùng Context khi	Chưa cần Context khi
Theme/user/locale/app config cho nhiều descendant.	Dữ liệu chỉ dùng trong một component.
Prop drilling dài và intermediate component chỉ forward dữ liệu.	Cha truyền thẳng xuống con một hoặc hai tầng rõ ràng.
Có boundary/provider hợp lý.	Chỉ muốn tránh viết một prop vì “ngại”.

7. useRef – DOM reference và mutable value không re-render

useRef trả về một object có thuộc tính current. Thay đổi ref.current không tự yêu cầu React render lại. Vì vậy useRef phù hợp cho DOM node, interval/timeout id hoặc mutable value không dùng trực tiếp để quyết định UI.

Focus input

```

import { useRef } from 'react';

function SearchBox() {
  const inputRef = useRef(null);

  function focusSearch() {
    inputRef.current?.focus();
  }

  return (
    <>
      <input ref={inputRef} />
      <button onClick={focusSearch}>Focus Input</button>
    </>
  );
}

```

```
}
```

Lưu timer id

```
const timerRef = useRef(null);

function start() {
  if (timerRef.current) return;
  timerRef.current = setInterval(() => {
    setSeconds(s => s + 1);
  }, 1000);
}

function stop() {
  clearInterval(timerRef.current);
  timerRef.current = null;
}
```

Đặc điểm	useState	useRef
Thay đổi gây re-render	Có (qua setter).	Không.
Dùng để quyết định UI	Có.	Không nên dùng ref làm nguồn UI cần render.
DOM reference	Không trực tiếp.	Use case phổ biến.
Mutable value qua render	Có nhưng update theo React.	Có qua ref.current.
Ví dụ	query, isOpen, products	input DOM, timer id, previous value

Cảnh báo — Không dùng ref như một “state bí mật” để né re-render. Nếu thay đổi giá trị phải làm UI đổi theo, dùng state.

8. Custom Hook – Tái sử dụng stateful logic

Custom Hook là hàm bắt đầu bằng use và có thể gọi các Hook khác. Custom Hook chia sẻ logic, không tự động chia sẻ cùng một state instance giữa các component.

useCounter

```
import { useState } from 'react';

export function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);

  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);
  const reset = () => setCount(initial);

  return { count, increment, decrement, reset };
}
```

useLocalStorage – mở rộng có kiểm soát

```
import { useEffect, useState } from 'react';

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const raw = localStorage.getItem(key);
```

```

    return raw ? JSON.parse(raw) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

```

Lưu ý: localStorage là browser persistence, không phải database/backend. Production code cần cân nhắc parse errors, schema/version, privacy và SSR/browser availability.

9. Rules of Hooks

Rule	Đúng	Sai
Call Hooks at top level	const [x,setX] = useState(0) ở đầu component.	Gọi Hook trong if/for/while/nested function.
Call Hooks from React functions	Function component hoặc custom Hook.	Regular helper function, event handler, class component.
Stable call order	Mỗi render đi qua Hook calls theo cùng trật tự.	Conditional Hook khiến thứ tự thay đổi.

```

// ❌ Sai
if (loggedIn) {
  const [name, setName] = useState('');
}

```

```

// ✅ Đúng
const [name, setName] = useState('');
if (!loggedIn) {
  return <LoginPrompt />;
}

```

Tooling — eslint-plugin-react-hooks có thể phát hiện vi phạm Rules of Hooks và dependency issues. Sinh viên vẫn phải hiểu lý do, không chỉ “sửa đến khi lint hết đỏ”.

10. Những hiểu sai phổ biến

Hiểu sai	Sửa mental model
Mọi data phải là state.	Chỉ state hóa dữ liệu cần nhớ và ảnh hưởng UI; derived value tính trực tiếp khi có thể.
useEffect cho mọi logic.	Effect dành cho synchronization với external system; event logic ở handler, derived logic ở render.
useRef thay useState được.	Ref không trigger re-render.
Context tốt hơn props.	Props là lựa chọn mặc định khi flow trực tiếp; Context có chi phí coupling/re-render.
Custom Hook là Component.	Hook trả logic/value; component trả UI.
Hook có thể gọi trong if.	Hook order phải ổn định.
Effect [] “chắc chắn chạy đúng 1 lần”.	Development StrictMode có thể chạy thêm setup/cleanup để kiểm tra; production semantics khác.
useState lưu vào backend.	useState là client-side component state.

11. Human Verification Gate

- Giải thích bằng lời: khi nào dùng state, effect, context, ref?
- Chỉ ra external system trong một Effect cụ thể.
- Giải thích dependency array của Effect đó.

- Chỉ ra cleanup và resource nó giải phóng.
- Giải thích vì sao một Hook call trong if là sai.
- Giải thích custom Hook chia sẻ logic chứ không tự động chia sẻ state instance.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM THỰC TẾ

1. Mapping Hook theo loại sản phẩm

Miền	useState	useEffect	useContext	useRef / Custom Hook
E-commerce	cart quantity, filter, form	sync title/network preview	current user/theme	focus search; useCart/useLocalStorage
Management System	form/modal/selection	API synchronization later	auth/locale	focus invalid input; useForm/useFetch later
Learning App	progress/filter/quiz state	timer/localStorage/network	current learner/theme	timer id; useProgress
Banking/Finance	form, selected account	secure network/session sync later	authenticated user/config	focus OTP input; auth custom hook

2. Sáu tình huống thực tế

- Tình huống 1: Tìm kiếm sản phẩm: useState lưu query; filtered list là derived value, không nhất thiết là state.
- Tình huống 2: Component cần đồng bộ document.title: useEffect với dependency đúng.
- Tình huống 3: Timer/auto refresh: Effect setup resource và cleanup resource.
- Tình huống 4: Theme dark/light toàn app: Provider sở hữu theme; children dùng useContext.
- Tình huống 5: Focus ô search khi người dùng nhấn nút: useRef tới DOM input.
- Tình huống 6: Nhiều component cần logic counter/timer giống nhau: tách custom hook; mỗi caller có state instance riêng.

3. Case study – Product Hook Demo Application

App

```

├─ Header          → useContext(theme)
├─ SearchBox       → useState(query) + useRef(input)
├─ ProductForm     → useState(form)
├─ ProductList     → props + map()
├   └─ ProductItem
└─ Footer
  
```

Shared logic:

useLocalStorage(products) / useCounter(optional)

Effect:

products.length → document.title

Concern	Owner/Hook	Lý do
products	App / useState hoặc useLocalStorage	Source of truth danh sách.
query	App hoặc SearchBox / useState	Tùy component nào cần filter.
filteredProducts	Derived value	Không cần duplicated state.
theme	ThemeProvider / useState + Context	Nhiều descendant cần.
search DOM	SearchBox / useRef	DOM imperative focus.
document.title	App / useEffect	External browser API synchronization.

4. Bài học kiến trúc

Trong sản phẩm thật, Hook không phải mục tiêu cuối. Mục tiêu là chọn đúng cơ chế cho đúng concern: state ownership rõ, effects ít và có lý do, context có boundary, refs không làm source of truth cho UI, custom hook chỉ tách logic khi có giá trị tái sử dụng hoặc làm component dễ hiểu hơn.

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN VIỆC THỰC HIỆN VÀ TRIỂN KHAI

A. Vì sao cần Hook? Function Component vs Class Component

Mục tiêu: sinh viên nhìn thấy Hook giải quyết bài toán cụ thể thay vì học thuộc lịch sử React.

A1. Hoạt động Predict → Compare → Explain

1. Cho sinh viên đọc Counter class và Counter function + useState.
2. Dự đoán nơi state được lưu, cách cập nhật và cách UI đọc state.
3. So sánh số khái niệm phải nhớ: this/state/setState/render so với state/setter.
4. Kết luận: Hook mở rộng function component; không phải “Hook thay Function Component”.

B. useState – Quản lý trạng thái

B1. Lab mini – Counter

```
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <section>
      <h3>Count: {count}</h3>
      <button onClick={() => setCount(c => c + 1)}>Increase</button>
      <button onClick={() => setCount(c => c - 1)}>Decrease</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </section>
  );
}
```

Checkpoint	PASS khi
Initial	Count hiển thị 0.
Increase	Mỗi click tăng đúng 1.
Decrease	Mỗi click giảm đúng 1.
Reset	Trở về 0.
Explain	Sinh viên nói được state snapshot + setter + re-render.

B2. Lab mini – Controlled Form

```
function StudentName() {
  const [name, setName] = useState('');
  return (
    <>
      <input value={name} onChange={e => setName(e.target.value)} />
      <button onClick={() => setName('')}>Clear</button>
      <p>Hello {name || 'student'}</p>
    </>
  );
}
```

```
);
}
```

Bài tập nhỏ: thêm email; không gộp mọi field vào object nếu điều đó làm bài khó hơn. Sau đó làm phiên bản object state để so sánh.

B3. Array state – immutable update

```
const [products, setProducts] = useState([
  { id: 1, name: 'Keyboard' },
  { id: 2, name: 'Mouse' }
]);

setProducts(prev => [
  ...prev,
  { id: Date.now(), name: newName }
]);
```

C. useEffect – Side effect, dependency và cleanup

C1. Mental trace

```
Render/Commit
↓
Effect setup
↓
External system synchronized
↓
Dependency changes?
↓ yes
Cleanup previous setup
↓
Run new setup
↓
Unmount → final cleanup
```

C2. Document title experiment

1. Chạy Counter không Effect; quan sát tab title không phản ánh count.
2. Thêm useEffect với [count].
3. Predict: click Increase thì dependency nào thay đổi?
4. Verify title và Browser Console.

C3. Timer experiment

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    return () => clearInterval(id);
  }, [seconds]);
}
```

```
}, []);
```

```
return <p>{seconds}s</p>;
}
```

Break It — Tạm bỏ `clearInterval`. Toggle component mount/unmount và quan sát behavior/console. Sau đó phục hồi `cleanup`. Mục đích: hiểu `cleanup` bằng evidence, không học thuộc.

C4. StrictMode awareness

Trong development, React StrictMode có thể chạy thêm một chu kỳ setup + cleanup trước setup thật để phát hiện Effect không đối xứng. Vì vậy đừng kết luận “[] luôn chạy đúng một lần trong mọi môi trường”. Thiết kế Effect sao cho setup và cleanup có thể lặp an toàn.

D. useContext – Chia sẻ dữ liệu

D1. ThemeProvider

```
// main.jsx hoặc App root
<ThemeProvider>
  <App />
</ThemeProvider>

// descendant
const { theme, toggleTheme } = useTheme();
```

D2. Context decision worksheet

Câu hỏi	YES/NO	Quyết định
Dữ liệu dùng bởi nhiều descendant ở xa?	—	Nếu NO, props/local state có thể đủ.
Intermediate component chỉ forward props?	—	Nếu YES, Context có thể đáng cân nhắc.
Dữ liệu thay đổi rất thường xuyên?	—	Cân nhắc phạm vi provider và re-render.
Có owner/provider rõ ràng?	—	Nếu NO, thiết kế ownership trước.

E. useRef – DOM và mutable reference

E1. Focus input sau hành động

```
const inputRef = useRef(null);

<input ref={inputRef} />
<button onClick={() => inputRef.current?.focus()}>
  Focus
</button>
```

E2. Auto-focus khi mount – kết hợp Effect có chủ đích

```
const inputRef = useRef(null);

useEffect(() => {
  inputRef.current?.focus();
}, []);

return <input ref={inputRef} />;
```

External system ở ví dụ này là DOM focus API. Đây là use case phù hợp cho ref + effect.

F. Custom Hook – Tái sử dụng logic

F1. Extract useCounter

1. Bắt đầu bằng Counter component hoạt động.
2. Đánh dấu phần logic state + handlers không phải JSX.
3. Chuyển logic đó vào useCounter(initial).
4. Component gọi useCounter và chỉ còn nhiệm vụ trình bày UI.
5. Tạo Counter thứ hai và xác minh hai component có state độc lập.

Điểm quan trọng — Custom Hooks share logic, not state. Hai component cùng gọi useCounter không tự động dùng chung một count.

G. Lab 01 – Working with ReactJS Application (integration)

Mục tiêu: tạo một artifact nhỏ nhưng chứng minh được kiến thức Slot 6. Nếu Lab 01 chính thức của lớp có schema/tài nguyên riêng, giữ nguyên yêu cầu chính thức; phần dưới là khung tích hợp Hook theo prompt Slot 6.

G1. Cấu trúc đề xuất

```
src/
├── components/
│   ├── Header.jsx
│   ├── Footer.jsx
│   ├── ProductForm.jsx
│   ├── ProductList.jsx
│   ├── ProductItem.jsx
│   ├── SearchBox.jsx
│   └── ThemeToggle.jsx
├── context/
│   └── ThemeContext.jsx
├── hooks/
│   ├── useCounter.js
│   └── useLocalStorage.js
├── App.jsx
└── main.jsx
```

G2. 10 bước triển khai

Bước	Thực hiện	PASS khi
1	Baseline	npm run dev; app cũ chạy; console sạch lỗi blocking.
2	Layout	Tạo Header/Main/Footer, verify từng component.
3	Static products	Tạo 3–5 records, render bằng map/key.
4	useState products	Đưa products thành state owner tại App hoặc Home feature.
5	ProductForm	Controlled inputs; submit thêm immutable.
6	Search	query state; filtered list là derived value.
7	useEffect	Đồng bộ document.title với products.length; giải thích external system.
8	useContext	ThemeProvider + toggle; chỉ theme shared.
9	useRef	Focus SearchBox/input bằng button hoặc mount effect.
10	Custom Hook + Evidence	Tách useLocalStorage hoặc useCounter; test, commit, README.

G3. Test checklist

Kiểm tra	Kết quả
App chạy	PASS / FAIL

Kiểm tra	Kết quả
Products render đúng	PASS / FAIL
Thêm product	PASS / FAIL
Xóa product	PASS / FAIL
Search filter	PASS / FAIL
Title cập nhật	PASS / FAIL
Theme toggle	PASS / FAIL
Focus input	PASS / FAIL
Rules of Hooks không vi phạm	PASS / FAIL
No blocking console error	PASS / FAIL
Git checkpoint	PASS / FAIL
README có cách chạy	PASS / FAIL

G4. Evidence bắt buộc

- Screenshot UI với ít nhất 5 products.
- Screenshot theme khác trạng thái mặc định.
- Browser Console không có blocking error.
- Đoạn giải thích: state owner, effect external system, context purpose, ref purpose.
- Git commit hash sau khi test PASS.

V. KINH NGHIỆM THỰC TIỄN, TIPS

Chủ đề	Lỗi thường gặp	Tip / Best practice
useState	Mutate trực tiếp; duplicated state.	Functional updater khi phụ thuộc state cũ; derive values khi có thể.
useEffect	Infinite loop; thiếu dependency; không cleanup.	Hỏi “external system nào?” trước khi viết Effect; setup/cleanup đối xứng.
useContext	Provider quá rộng; mọi thứ đưa vào context.	Giữ local state local; scope provider theo feature.
useRef	Dùng ref để cập nhật UI.	Nếu UI phải đổi → state; ref cho reference không render.
Custom Hook	Tách quá sớm; naming không use*.	Extract khi logic stateful có ý nghĩa tái sử dụng hoặc làm component dễ hiểu hơn.
AI Debug	Prompt mơ hồ: “code không chạy”.	Cung cấp Expected, Actual, first error, relevant code, last change.

Debugging decision tree

```

Hook-related problem
↓
Does app compile?
↓
Hook imported?
↓
Rules of Hooks respected?
↓
State: correct owner + setter?
↓
Effect: external system + dependencies?
↓
Cleanup symmetric?
↓
Context: Provider above consumer?
↓
Ref: current exists when used?
↓

```

Then inspect feature-specific logic

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

A. 12 bài tập chủ đề

Bài	Yêu cầu
1. useState – Counter	Increase / Decrease / Reset. Giải thích vì sao functional updater phù hợp.
2. useState – Form Input	Tên sinh viên, Hello {name}, Clear, controlled input.
3. useState – Product List	Thêm/xóa, tổng số, immutable update.
4. useEffect – Document Title	Count thay đổi → title cập nhật. Chỉ ra dependency.
5. useEffect – Fetch Users	Fetch JSONPlaceholder ở mức preview, loading/error/data; nêu boundary Slot 8/9.
6. useEffect – Timer	Start/Stop, cleanup interval khi unmount.
7. useContext – Theme	ThemeContext, Header/Content đọc theme.
8. useContext – User Session	Login/Logout giả lập; Header hiện user.
9. useRef – Focus Input	Focus on mount hoặc click.
10. useRef – Timer ID / Render Observation	Lưu mutable reference không gây render; giải thích vì sao không dùng ref làm UI source.
11. Custom Hook – useCounter	count, increment, decrement, reset.
12. Custom Hook – useLocalStorage	Lưu state browser-side; reload còn giá trị; nêu giới hạn.

B. Bài tập tổng hợp – React Hook Demo Application

Mục tiêu: xây một mini application đủ để chứng minh sinh viên có thể chọn và phối hợp Hook theo responsibility, không chỉ ghép nhiều Hook cho đủ checkbox.

Chức năng	Hook/Concept dự kiến	Tiêu chí kiểm chứng
Danh sách sản phẩm	useState + props + map	Records render đúng; key ổn định.
Form thêm	useState controlled form	Submit làm danh sách tăng; không mutation.
Tim kiếm	useState + derived filter	Không cần Effect để filter.
Tổng sản phẩm	Derived value	products.length; không state hóa lại.
Xóa	Event + state setter	Xóa đúng item.
Dark/Light	Context + useState	Nhiều component đọc theme.
Document title	useEffect	Title theo số sản phẩm.
Focus search	useRef	DOM focus hoạt động.
Persistence	Custom useLocalStorage – optional/extension	Reload giữ data và giải thích browser storage.
README/Git	Engineering evidence	Cách chạy + commit hash.

C. Challenge “You Might Not Need an Effect”

Cho code lưu filteredProducts vào state rồi useEffect cập nhật mỗi khi products/query đổi. Yêu cầu sinh viên refactor bằng `const filteredProducts = products.filter(...)` trong render. Giải thích vì sao version mới đơn giản và tránh thêm render/effect dependency.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ trong bài
Hook	Hàm React đặc biệt cung cấp khả năng React cho function component/custom hook.	useState(0)
Function Component	Hàm JavaScript mô tả UI React.	function Counter() {...}
Class Component	Component dựa trên class, dùng state/lifecycle APIs kiểu class.	class Counter extends React.Component
useState	State Hook tạo state variable + setter.	const [count, setCount]=useState(0)
useEffect	Effect Hook đồng bộ với external system.	document.title / timer / network
useContext	Đọc và subscribe Context.	useContext(ThemeContext)
useRef	Mutable reference không trigger re-render.	inputRef.current.focus()
Custom Hook	Hàm use* tái sử dụng stateful logic.	useCounter
Rules of Hooks	Quy tắc vị trí gọi Hook.	Top level; React functions only
State	Bộ nhớ component ảnh hưởng UI.	count/searchText
Setter	Hàm update state.	setCount
Re-render	React gọi component lại để tính UI mới.	sau setCount
Dependency array	Reactive values Effect phụ thuộc.	[count]
Cleanup	Hàm hoàn tác setup Effect.	clearInterval
Side effect	Synchronization với external system.	network/browser API/timer
Controlled component	Input có value từ state và onChange update state.	<input value={name} ...>

Thuật ngữ	Định nghĩa	Ví dụ trong bài
Context	Kênh dữ liệu qua tree.	ThemeContext
Provider	Boundary cung cấp context value.	ThemeContext.Provider
Consumer	Component đọc context value.	useTheme/useContext
Ref	Object mutable có current.	{ current: ... }
DOM reference	Ref trỏ DOM node.	input ref
Immutable update	Tạo object/array mới thay vì mutate cũ.	[...prev,newItem]
Lifecycle gốc nhìn Hook	Setup/sync/cleanup theo render/commit và dependency.	Effect setup + cleanup
Lifting state up	Đưa state lên common parent.	Search+List shared query
Single source of truth	Một owner chính cho dữ liệu thay đổi.	products owned by App
React tree	Cấu trúc component/provider descendant.	App→Header/List

VIII. RESEARCH BASE VÀ AI-BASED LEARNING

Nguyên tắc sử dụng AI: AI có thể giải thích, review, tạo giả thuyết và đề xuất thí nghiệm; sinh viên phải chạy code, thu thập evidence và tự giải thích. “AI may generate; student must verify.”

A. 10 prompt tự nghiên cứu chuyên sâu kỹ thuật

1. Phân tích cơ chế re-render khi useState setter được gọi. Phân biệt state snapshot, batching và functional updater. Đưa ví dụ nhỏ và yêu cầu tôi dự đoán output trước khi giải thích.
2. So sánh useState và useReducer về state shape, transition complexity, testability và khi nào nên chuyển từ useState sang reducer. Không mặc định reducer tốt hơn.
3. Phân tích dependency array trong useEffect. Cho 5 ví dụ đúng/sai và giải thích dependency nào là reactive value.
4. Giải thích closure và stale state trong React hooks bằng timeline render. Đưa ví dụ timer/event callback và cách sửa.
5. Phân tích cleanup function: khi nào chạy, vì sao setup/cleanup phải đối xứng, và StrictMode development giúp phát hiện lỗi gì.
6. So sánh props drilling với Context. Cho decision tree để quyết định props, lifting state hay Context.
7. So sánh useState và useRef với các ví dụ: UI value, DOM node, timer id, previous value.
8. Giải thích Rules of Hooks và vì sao call order phải ổn định. Phân tích 5 đoạn code vi phạm mà không sửa ngay, để tôi tự đoán.
9. Phân tích custom Hook: logic nào nên extract và logic nào không nên. Nhấn mạnh custom Hook share logic chứ không share state.
10. Đánh giá một component React dài và xác định boundary hợp lý để tạo custom hook, dựa trên responsibility và reuse evidence.

B. 10 prompt tự nghiên cứu thực tiễn chuyên nghiệp

1. Trong dự án React lớn, trình bày chiến lược state ownership: local state, lifted state, Context, server-state library. Cho tiêu chí thay vì chọn library theo trend.
2. Khi nào dùng Context, Redux Toolkit, Zustand hoặc Jotai? So sánh scope, update frequency, debugging và team conventions.
3. Mô tả pattern production cho loading/error/data khi gọi API. Phân biệt local UI state với server state.
4. Vì sao production app thường dùng TanStack Query/SWR thay vì tự fetch bằng useEffect? Nêu lợi ích caching, dedup, retries, invalidation và trade-off.
5. Làm thế nào test component/custom hook có useState/useEffect bằng React Testing Library? Tập trung behavior hơn implementation detail.
6. Authentication state trong SPA thường gồm gì? Phân biệt identity, access token, server session, Context và persistence.
7. Làm thế nào giảm unnecessary re-render do Context? Phân tích provider scope, value identity và context splitting.

8. Khi app có nhiều form, React Hook Form giải quyết vấn đề nào so với controlled inputs thuần? Nêu trade-offs.
9. Tạo checklist debug exhaustive-deps/useEffect dependency issues trong team React.
10. Đề xuất coding conventions cho custom hooks trong team 6 người: naming, return shape, error handling, testing, documentation.

C. 10 prompt khám phá xu hướng/cập nhật

1. Tra cứu tài liệu React.dev hiện tại và giải thích React Compiler là gì, nó tối ưu gì và không làm mất nhu cầu hiểu state/effect correctness như thế nào.
2. Tra cứu Server Components và phân tích component nào có thể/cannot use client Hooks. Phân biệt kiến trúc framework với SPA client-only của SBA301.
3. Tra cứu React 19 và lập bảng các API/khái niệm mới liên quan state/actions/hooks; đánh dấu core SBA301 vs professional awareness.
4. So sánh xu hướng state management: Context, Redux Toolkit, Zustand, Jotai, TanStack Query. Phân biệt client state và server state.
5. So sánh TanStack Query và SWR cho data fetching/caching; giải thích vì sao đây là nội dung sau khi đã hiểu effects/HTTP.
6. Tìm 3 cách tổ chức project React hiện đại (feature-based, domain-based, type-based) và đánh giá phù hợp với project SBA301 4–5 người.
7. Tổng hợp best practices useEffect mới nhất từ React.dev; đặc biệt “You Might Not Need an Effect”. Cho ví dụ refactor.
8. Phân tích lợi ích TypeScript khi dùng hooks: state union, context nullability, custom hook contracts.
9. Tra cứu Vite, ESLint React Hooks và React DevTools hiện tại; tạo checklist tooling cho sinh viên.
10. Tổng hợp 10 lỗi hiệu năng với hooks trong app lớn và phân loại: correctness trước, optimization sau.

D. AI Debug Prompt mẫu

Act as a React debugging tutor.
Do not rewrite my whole component.

Expected behavior: [EXPECTED]
Actual behavior: [ACTUAL]
First console error/warning: [ERROR]
Relevant state/hooks: [CODE]
Last change: [CHANGE]

Trace in this order:

1. Rules of Hooks
2. state ownership and setter
3. event handler
4. effect external system
5. dependencies
6. cleanup
7. context provider
8. ref.current timing

Identify the FIRST broken assumption and give one verification step.

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Kiểm tra kiến thức nền tảng

- Hook là gì? Hook có thay function component không?
- Vì sao function component hiện đại dùng Hook?
- useState trả về gì? Khi nào dùng functional updater?
- Side effect là gì? External system trong ví dụ của bạn là gì?
- Dependency array ảnh hưởng Effect ra sao?
- Cleanup chạy để làm gì?
- Vì sao không nên dùng Effect để filter list nếu có thể derive?
- useContext khác props như thế nào?
- useRef khác useState như thế nào?
- Custom Hook share logic hay share state?
- Hai Rules of Hooks là gì?

2. Kiểm tra kỹ năng thực hành

Kỹ năng	Đạt	Chưa đạt	Evidence
Tạo/useState đúng	☐	☐	_____
Immutable update object/array	☐	☐	_____
Controlled input	☐	☐	_____
Effect với dependency đúng	☐	☐	_____
Cleanup timer/listener	☐	☐	_____
Theme Context + Provider	☐	☐	_____
useRef focus input	☐	☐	_____
Custom hook useCounter/useLocalStorage	☐	☐	_____
Không vi phạm Rules of Hooks	☐	☐	_____
Lab 01 chạy và test PASS	☐	☐	_____

3. Mức độ tự đánh giá

Mức	Dấu hiệu
Mức 1 – Chưa đạt	Chưa phân biệt state/effect/ref; chưa chạy được useState/useEffect demo; Lab chưa ổn.
Mức 2 – Đạt cơ bản	Giải thích được Hook; dùng useState/useEffect cơ bản; Context/ref cần hỗ trợ; Lab chạy.
Mức 3 – Tốt	Chọn đúng Hook theo responsibility; effect có dependency/cleanup hợp lý; context/ref có lý do; custom hook đơn giản; giải thích và debug được.

4. Final Human Verification – 15 câu không AI

1. Vì sao biến thường thay đổi không đủ để cập nhật UI React?
2. State khác ref ở điểm quyết định nào?
3. Khi nào một value nên là derived value thay vì state?
4. Effect đang đồng bộ với external system nào?
5. [] có nghĩa gì và development StrictMode có lưu ý gì?
6. Cleanup cần cho interval/listener ra sao?
7. Một dependency bị bỏ sót có thể tạo bug gì?
8. Props, lifting state và Context: ưu tiên chọn theo thứ tự reasoning nào?
9. Provider nằm dưới consumer thì điều gì xảy ra?
10. Vì sao ref.current thay đổi không làm UI tự cập nhật?
11. Custom Hook có thể gọi Hook khác không?

12. Custom Hook có chia sẻ cùng state giữa callers không?
13. Gọi useState trong if sai vì sao?
14. Tại sao map() không phải Hook?
15. React Hook chuẩn bị cho Router/HTTP/data fetching các slot sau như thế nào?

5. Việc cần làm nếu chưa đạt

- Làm lại Counter bằng useState và giải thích từng dòng.
- Làm controlled input; phân biệt state với biến thường.
- Làm document.title Effect và viết đúng dependency.
- Làm interval có cleanup, sau đó thử “break it”.
- Đọc lại Rules of Hooks và tự tìm 3 vi phạm trong code mẫu.
- Làm ThemeContext nhỏ; giải thích vì sao Context được dùng.
- Làm focus input bằng useRef.
- Hoàn thiện Lab 01 tối thiểu đến mức test checklist PASS và có Git commit.

PHỤ LỤC A. QUALITY GATE CHO SLOT 6

Tiêu chí	Kết quả
Đúng Slot 6 – React Hook + Lab 01	PASS
Có tóm tắt Slot 5 và định vị các slot sau	PASS
Giải thích Why Hooks + Function vs Class	PASS
useState: primitive/object/array/controlled form	PASS
useEffect: side effect, dependencies, cleanup	PASS
Có “You Might Not Need an Effect” awareness	PASS
useContext: Provider + useContext + anti-overuse	PASS
useRef: DOM + mutable value không re-render	PASS
Custom Hook: useCounter/useLocalStorage	PASS
Rules of Hooks + invalid examples	PASS
Lab 01 integration + testing evidence	PASS
Common errors + best practices + debugging tree	PASS
12 bài tập + bài tổng hợp	PASS
Glossary	PASS
30 AI/Research prompts (3 nhóm)	PASS
Self-check + Human Verification Gate	PASS

PHỤ LỤC B. QUICK REFERENCE

Need	Choose first	Câu hỏi tự hỏi
UI needs remembered value	useState	Value này có đổi và ảnh hưởng render?
Synchronize external system	useEffect	External system nào? Setup/cleanup là gì?
Distant shared value	useContext	Props drilling có thực sự xảy ra?
DOM/mutable non-render value	useRef	UI có cần re-render khi giá trị đổi?
Reuse stateful logic	Custom Hook	Logic này có repeated purpose rõ ràng?